

Plan de Branching - Parcial 2

Asignatura: DSY1106 - Desarrollo Fullstack III
Proyecto: Sanos y Salvos

1. Objetivo

Definir una estrategia de ramas que facilite trabajo paralelo de frontend/backend, control de versiones y trazabilidad de integraciones.

2. Estrategia propuesta (GitFlow liviano)

Ramas principales

- `main`: rama estable, lista para entrega/despliegue.
- `develop` (opcional en iteraciones futuras): rama de integracion continua del equipo.

Ramas de trabajo

- `feature/<nombre-corto>`: nuevas funcionalidades.
- `fix/<nombre-corto>`: correcciones de bugs.
- `docs/<nombre-corto>`: mejoras documentales.
- `chore/<nombre-corto>`: tareas tecnicas internas (build, scripts, refactor menor).

3. Flujo de trabajo recomendado

1. crear rama desde `main` (o `develop` cuando se use).
2. hacer commits pequenos y descriptivos.
3. abrir Pull Request.
4. revisar cambios y resolver conflictos.
5. merge a `main` con historial trazable.

4. Convencion de commits

Se recomienda convencional:

- `feat`: nueva funcionalidad
- `fix`: correccion
- `docs`: documentacion
- `test`: pruebas
- `chore`: tareas de soporte

5. Gestion de conflictos

- actualizar rama local antes de PR (`git pull --rebase` o merge controlado).
- resolver conflicto manteniendo consistencia funcional y de estilos.
- ejecutar pruebas unitarias y build Docker luego de resolver.
- dejar evidencia en PR/comentarios tecnicos.

6. Evidencia actual en repositorio

Se observa historial de integraciones y al menos un merge registrado en `main`:

- commit merge: 80242be
- commits funcionales y de pruebas en cadena sobre `main`.

7. Plan de mejora para siguientes iteraciones

- institucionalizar uso de `develop`.
- crear PR templates con checklist (tests, docker build, docs).
- proteger `main` con regla de review minima y estado de checks.
- agregar workflow CI para tests y build automatico en PR.